

1. Отчет по тестированию индексов в PostgreSQL

1.1. Введение

В данном отчете представлены результаты тестирования различных типов индексов в PostgreSQL на базе данных ресторана. Для каждого теста приведены: код запроса, план выполнения до создания индекса, SQL-код индекса и план выполнения после создания индекса.

1.2. Простые индексы

1.2.1. Код запроса

```
SELECT last_name FROM guests WHERE last_name = 'Иванов';
```

1.2.2. Выполнение без индексов

1.2.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_guests_lastname;  
CREATE INDEX idx_guests_lastname ON guests USING BTREE (last_name);
```

1.2.4. Выполнение с индексами B-Tree

1.2.5. Код индекса Hash

```
DROP INDEX IF EXISTS idx_guests_lastname_hash;  
CREATE INDEX idx_guests_lastname_hash ON guests USING HASH (last_name);
```

1.2.6. Выполнение с индексами Hash

1.3. Уникальный индекс

1.3.1. Код запроса

```
SELECT table_number FROM tables WHERE table_number = 5;
```

1.3.2. Выполнение без индексов

1.3.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_tables_tablenumber_unique;  
CREATE UNIQUE INDEX idx_tables_tablenumber_unique ON tables USING BTREE  
(table_number);
```

1.3.4. Выполнение с индексами B-Tree

1.4. Составной индекс

1.4.1. Код запроса

```
SELECT guest_id, order_time FROM orders WHERE guest_id = 100 AND order_time >=  
'2025-11-01';
```

1.4.2. Выполнение без индексов

1.4.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_orders_guest_time;  
CREATE INDEX idx_orders_guest_time ON orders USING BTREE (guest_id,  
order_time);
```

1.4.4. Выполнение с индексами B-Tree

1.5. Индексы с использованием выражений

1.5.1. Код запроса

```
SELECT * FROM guests WHERE DATE_PART('year', birth_date) = 1990;
```

1.5.2. Выполнение без индексов

1.5.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_guests_birth_year;  
CREATE INDEX idx_guests_birth_year ON guests USING BTREE (DATE_PART('year',  
birth_date));
```

1.5.4. Выполнение с индексами B-Tree

1.5.5. Код индекса Hash

```
DROP INDEX IF EXISTS idx_guests_birth_year_hash;  
CREATE INDEX idx_guests_birth_year_hash ON guests USING HASH  
(DATE_PART('year', birth_date));
```

1.5.6. Выполнение с индексами Hash

1.6. Покрывающий индекс

1.6.1. Код запроса

```
SELECT name, price FROM dishes WHERE category = 'Main';
```

1.6.2. Выполнение без индексов

1.6.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_dishes_category_covering;  
CREATE INDEX idx_dishes_category_covering ON dishes USING BTREE (category)  
INCLUDE (name, price, country_of_origin);
```

1.6.4. Выполнение с индексами B-Tree

1.7. Частичный индекс

1.7.1. Код запроса

```
SELECT status, order_time FROM orders WHERE status = 'paid' AND order_time >=  
'2025-11-01';
```

1.7.2. Выполнение без индексов

1.7.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_orders_paid;  
CREATE INDEX idx_orders_paid ON orders USING BTREE (order_time)  
WHERE status = 'paid';
```

1.7.4. Выполнение с индексами B-Tree

1.7.5. Код индекса Hash

```
DROP INDEX IF EXISTS idx_orders_paid_status_hash;  
CREATE INDEX idx_orders_paid_status_hash ON orders USING HASH (guest_id)  
WHERE status = 'paid';
```

1.7.6. Выполнение с индексами Hash

1.8. Частичный покрывающий индекс

1.8.1. Код запроса

```
SELECT guest_id, table_id, total_amount FROM orders  
WHERE status = 'paid' AND order_time >= '2025-11-01';
```

1.8.2. Выполнение без индексов

1.8.3. Код индекса B-Tree (частичный)

```
DROP INDEX IF EXISTS idx_orders_paid_partial;  
CREATE INDEX idx_orders_paid_partial ON orders (order_time)  
WHERE status = 'paid';
```

1.8.4. Выполнение с частичным индексом

1.8.5. Код индекса B-Tree (частичный покрывающий)

```
DROP INDEX IF EXISTS idx_orders_paid_partial_covering;  
CREATE INDEX idx_orders_paid_partial_covering ON orders (order_time)  
INCLUDE (guest_id, table_id, total_amount)  
WHERE status = 'paid';
```

1.8.6. Выполнение с частичным покрывающим индексом

1.9. Дополнительные типы запросов

1.9.1. Запросы с соединениями таблиц

1.9.1.1. JOIN с составным индексом

```
SELECT g.last_name, g.first_name, o.order_time, o.total_amount  
FROM guests g  
JOIN orders o ON g.id = o.guest_id  
WHERE g.last_name = 'Иванов' AND o.order_time >= '2025-11-01';
```

1.9.1.2. JOIN с покрывающим индексом

```
SELECT d.name, d.price, oi.quantity  
FROM dishes d
```

```
JOIN order_items oi ON d.id = oi.dish_id
WHERE d.category = 'Main';
```

1.9.2. Фильтрация с предикатами

1.9.2.1. EXISTS

```
SELECT guest_id, status FROM guests g
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.guest_id = g.id AND o.status = 'paid'
);
```

1.9.2.2. IN

```
SELECT category FROM dishes
WHERE category IN ('Main', 'Dessert', 'Starter');
```

1.9.2.3. BETWEEN

```
SELECT order_time FROM orders
WHERE order_time BETWEEN '2025-11-01' AND '2025-11-30';
```

1.9.3. Функции для работы со строками

1.9.3.1. SUBSTRING

```
DROP INDEX IF EXISTS idx_dishes_name_substr;
CREATE INDEX idx_dishes_name_substr ON dishes (SUBSTRING(name, 1, 5));
```

```
SELECT name FROM dishes
WHERE SUBSTRING(name, 1, 5) = 'Цезар';
```

1.9.4. Функции даты и времени

1.9.4.1. DATE_TRUNC с временной зоной

```
SELECT date_trunc('hour', order_time AT TIME ZONE 'Europe/Moscow') AS
order_hour,
    COUNT(*) as orders_count
FROM orders
WHERE order_time >= '2025-11-01'
GROUP BY order_hour;
```

1.9.5. Агрегатные функции и GROUP BY

1.9.5.1. Агрегация с частичным покрывающим индексом

```
SELECT guest_id, COUNT(*) as order_count, SUM(total_amount) as total_spent
FROM orders
WHERE status = 'paid'
GROUP BY guest_id
HAVING SUM(total_amount) > 5000;
```

1.9.6. Вложенные запросы

1.9.6.1. Коррелированный подзапрос

```
SELECT g.last_name, g.first_name,
    (SELECT COUNT(*) FROM orders o
```

```
WHERE o.guest_id = g.id AND o.status = 'paid') as paid_orders  
FROM guests g  
WHERE total_orders > 10;
```

1.9.7. UNION и INTERSECT

1.9.7.1. UNION с покрывающими индексами

```
SELECT name, price FROM dishes WHERE category = 'Main'  
UNION  
SELECT name, price FROM dishes WHERE category = 'Dessert';
```

1.9.7.2. INTERSECT с частичными индексами

```
SELECT guest_id FROM orders WHERE status = 'paid' AND order_time >=  
'2025-11-01'  
INTERSECT  
SELECT guest_id FROM orders WHERE status = 'paid' AND total_amount > 2000;
```

1.10. Выводы

В ходе тестирования были продемонстрированы следующие типы индексов и их эффективность:

1. Простые индексы (B-Tree и Hash) - значительно ускоряют поиск по одному столбцу.
2. Уникальные индексы - обеспечивают уникальность данных и быстрый поиск.
3. Составные индексы - эффективны для запросов с условиями по нескольким столбцам.
4. Индексы с выражениями - позволяют индексировать результаты функций.
5. Покрывающие индексы - обеспечивают Index Only Scan, исключая обращения к таблице.
6. Частичные индексы - уменьшают размер индекса и ускоряют запросы с постоянным условием.
7. Частичные покрывающие индексы - комбинируют преимущества частичных и покрывающих индексов.

Все типы индексов показали значительное улучшение производительности по сравнению с последовательным сканированием таблиц.